

A PROJECT BASED SEMINAR REPORT

on

TextToAutoML

Natural Language Driven Machine Learning Automation

Submitted towards the
Partial Fulfilment of the Requirements of

**Bachelor of Technology in
Artificial Intelligence and Data Science**

By

Manasi Umesh Jadhav Seat No: T231311039

Under the guidance of

Dr. S. A. Khaire



Department of Artificial Intelligence and Data Science

K. K. WAGH EDUCATION SOCIETY

K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH

Hirabai Haridas Vidyanagari, Amrutdham, Panchavati, Nashik, Maharashtra 422003

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

A. Y. 2025-2026 Sem II



Department of Artificial Intelligence and Data Science
K. K. WAGH EDUCATION SOCIETY
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
(An Autonomous Institute affiliated to Savitribai Phule Pune University)

CERTIFICATE

This is to certify that the seminar report entitled

TextToAutoML

Natural Language Driven Machine Learning Automation

being submitted by

Manasi Umesh Jadhav Roll No: T231311039

is a bonafide work carried out by students under the supervision of Dr. S. A. Khaire and it is submitted towards the partial fulfilment of the requirement of Bachelor of Technology (Artificial Intelligence and Data Science) during academic year 2024-2025.

Dr. S. A. Khaire
Internal Guide
Department of
Artificial Intelligence and Data Science

Head,
Department of
Artificial Intelligence and Data Science

Abstract

Despite rapid advances in automated machine learning, a critical gap persists at the human-system interface: most non-technical users cannot interact with AutoML platforms because they require structured inputs and domain-specific terminology. This paper presents Text-ToAutoML, a natural language-driven system that eliminates this barrier by enabling users to describe their machine learning problem in plain English and automatically constructing, training, and optimizing a complete ML pipeline. The primary contribution of this work is the NLP and Intent Recognition module, which fine-tunes transformer-based models—BERT and RoBERTa—to classify user problem descriptions into structured ML task intents spanning classification, regression, and clustering. The module is supported by a text preprocessing and tokenization pipeline that normalizes and encodes diverse, informal user inputs, and a clarification prompt generation mechanism that detects and resolves ambiguous queries before pipeline execution. Unlike systems that rely on technically phrased prompts, Text-ToAutoML accepts raw, informal descriptions from non-experts. The recognized task intent is forwarded to an AutoML backend employing meta-learning for model recommendation and automated frameworks for hyperparameter optimization and evaluation. The system achieves over 90% task-type identification accuracy and substantially reduces ML development time, offering a scalable, domain-agnostic solution for healthcare, business analytics, education, and governance.

Keywords: *Natural Language Processing, Intent Recognition, Transformer Fine-Tuning, BERT, RoBERTa, Task Mapping Schema, Text Preprocessing, Tokenization, Clarification Prompt Generation, AutoML, Meta-Learning, No-Code AI, AI Democratization*

Acknowledgement

I would like to thank my project guide, Dr. S. A. Khair, for her guidance and support. I will forever remain grateful for the constant support and guidance extended by my guide, in making this project phase I report. Through our many discussions, she helped me to form and solidify ideas. The invaluable discussions I had with her and my team colleagues, the penetrating questions she has put to me and the constant motivation, has all led to the development of the ideas presented in this project phase I report.

I would also like to thank my wonderful colleagues, parents and friends for listening my ideas, asking questions and providing feedback and suggestions for improving my ideas.

Manasi Umesh['] Jadhav

Contents

1	Introduction	1
1.1	Project Idea	1
1.2	Motivation	2
1.3	Literature Survey	3
2	Problem Definition and Scope	8
2.1	Problem Statement	8
2.1.1	Goals and Objectives	8
2.1.2	Assumptions and Scope	9
3	Methodology	11
3.1	Methodology	11
4	Outcome	15
4.1	Type of Project	15
5	Project Plan	17
5.1	Project Timeline	17
5.2	Team Organization	17
5.2.1	Team Structure	18
6	Experimental Setup	20
6.1	Data Set	20
6.2	Technology Used	21
6.3	Performance Parameters	21
7	PO & PSO Mapping	23
7.1	Program Outcome (PO) Mapping	23
7.2	Program Specific Outcome (PSO) Mapping	26
8	Summary	27
	References	28

List of Figures

3.1	Natural Language to Machine Learning Model	12
3.2	Working Mechanism	14

List of Tables

5.1	Project Timeline	17
7.1	Program Outcomes (PO) Mapping – TextToAutoML	23
7.2	Program Specific Outcomes (PSO) Mapping – TextToAutoML	26

Chapter 1

Introduction

1.1 Project Idea

Machine learning has become one of the most powerful tools for solving complex real-world problems across diverse domains, including healthcare, finance, education, and government. However, despite its immense potential, the adoption of machine learning remains limited to organizations and individuals who possess deep technical expertise. Building an effective ML pipeline requires knowledge of data preprocessing, feature engineering, model selection, hyperparameter tuning, and performance evaluation—a skill set that most domain experts and non-technical users simply do not have.

TextToAutoML addresses this critical barrier by providing an intelligent system that allows users to describe their machine learning problem in simple, everyday natural language. The system then automatically interprets this description, identifies the appropriate ML task, selects the best-suited model, and constructs a fully optimized machine learning pipeline—without requiring any coding expertise from the user.

The core technologies that power TextToAutoML include:

- **Natural Language Processing (NLP):** To parse and understand user-provided problem descriptions. Transformer-based NLP models analyze the intent behind the text and map it to a structured machine learning task definition.
- **Meta-Learning:** To leverage prior knowledge about the performance of different ML algorithms on similar datasets. Meta-learning enables the system to intelligently recommend the most suitable models without exhaustively searching all options.
- **AutoML Frameworks:** To automate the entire ML pipeline, including data preprocessing, feature selection, model training, and hyperparameter optimization. Tools such as Auto-Sklearn and TPOT are utilized for this purpose.

The system aims to create a seamless experience for users, abstracting away all technical complexities of the ML lifecycle. Users simply describe their problem—for example,

“predict customer churn from sales data” or “classify patient records by disease type”—and TextToAutoML handles everything from data understanding to final model delivery.

By democratizing machine learning, TextToAutoML empowers businesses, students, healthcare professionals, and government analysts to leverage the power of AI without depending on scarce and expensive data science talent. The output of the system can be directly integrated into existing workflows, making it both practical and impactful across multiple sectors.

1.2 Motivation

The rapid growth of data across industries has created an unprecedented demand for machine learning solutions. However, the supply of skilled data scientists and ML engineers remains far behind this demand. Organizations—especially small and medium enterprises, healthcare institutions, and government bodies—often struggle to hire the technical talent required to build and maintain ML systems. This creates a significant gap between the potential and the actual utilization of AI in practice.

The motivation behind the development of TextToAutoML stems from addressing this fundamental inequality in AI access. The following key factors motivated this project:

1. **Democratizing Machine Learning:** The majority of potential ML users—including doctors, business managers, teachers, and policymakers—have domain knowledge but lack programming skills. TextToAutoML empowers these users to build ML models simply by describing their problems in natural language.
2. **Reducing Dependency on Experts:** Data science expertise is costly and scarce. By automating the end-to-end ML pipeline, TextToAutoML significantly reduces the dependency on specialized data scientists, making AI accessible to a much wider audience.
3. **Accelerating Time-to-Insight:** Traditional ML development cycles take weeks or months. TextToAutoML can generate, train, and evaluate a model in minutes, dramatically accelerating the time from problem identification to actionable insights.
4. **Minimizing Human Error:** Manual ML pipeline construction is prone to errors in model selection, data preprocessing, and hyperparameter tuning. Automating these steps with intelligent algorithms ensures more consistent and reliable results.
5. **Enabling Non-Technical Innovation:** Many domain experts have valuable ideas but cannot implement them due to technical barriers. TextToAutoML removes these barriers, allowing domain knowledge to directly drive AI-powered solutions.
6. **Bridging NLP and AutoML:** Existing AutoML tools still require users to have some technical understanding of the input format and task type. Integrating NLP as the user interface creates a truly natural and intuitive interaction paradigm.

7. **Scalability Across Domains:** The system is designed to be domain-agnostic, supporting use cases in healthcare diagnostics, retail forecasting, educational analytics, and government policy analysis, among others.

The motivation behind TextToAutoML is rooted in the belief that AI should be inclusive—accessible not just to technical experts, but to anyone with a problem to solve and data to analyze. By combining state-of-the-art NLP with meta-learning and AutoML, this project seeks to make that vision a reality.

Ultimately, TextToAutoML aspires to reduce the technical barrier of entry into machine learning, foster innovation across non-technical domains, and contribute to a future where AI is a universal tool rather than an exclusive privilege.

1.3 Literature Survey

The integration of Natural Language Processing (NLP) with automated machine learning (AutoML) pipelines has emerged as a significant research direction aimed at democratizing artificial intelligence. Recent works highlight that while AutoML has achieved considerable success in automating model development, challenges related to usability, adaptability, and end-to-end automation persist. This section presents a detailed analysis of key contributions that influence the design of TextToAutoML, focusing on usability limitations of AutoML platforms, LLM-driven orchestration of ML pipelines, natural language-to-pipeline translation, transformer-based intent classification, and human-centered multimodal AutoML frameworks.

Sakpere et al. [2] conducted an empirical, user-centered study involving 38 participants to evaluate the usability and adoption challenges associated with AutoML platforms. The study employed quantitative metrics such as task completion rate, learning progression over time, system interaction efficiency, and user perception analysis. Their findings reveal a critical contradiction: although AutoML is designed to abstract complexity, users still encounter significant barriers in understanding workflow configuration, selecting appropriate preprocessing techniques, and interpreting model outputs. The study highlights that many platforms require implicit technical knowledge, particularly in areas such as feature selection, parameter tuning, and evaluation metrics, which limits accessibility for non-experts. Additionally, the authors emphasize deficiencies in human-computer interaction (HCI), noting that current interfaces often lack guided assistance, contextual explanations, and adaptive feedback mechanisms. A particularly important observation is that even users who initially perceived AutoML tools as easy to use experienced performance degradation when dealing with real-world tasks, indicating hidden system complexity. The authors conclude that improving usability requires integrating intelligent interfaces capable of understanding user intent and reducing cognitive overhead. TextToAutoML directly addresses this limitation by introducing a natural language interface that eliminates the need for manual configuration

and provides an intuitive interaction paradigm, thereby aligning system design with user expectations identified in this study.

Veerapaneni et al. [6] proposed a self-evolving machine learning infrastructure that utilizes Large Language Models (LLMs) as intelligent orchestration agents for managing and optimizing heterogeneous data pipelines in cloud-based environments. Their architecture is composed of multiple layers, including monitoring modules, reasoning engines, and execution controllers, which together enable autonomous pipeline management. The system continuously monitors pipeline states, detects anomalies such as schema drift, missing values, and data inconsistencies, and generates corrective actions without human intervention. By incorporating LLMs into the control loop, the system demonstrates the ability to perform contextual reasoning over pipeline states, making decisions based on both historical patterns and real-time data conditions. This closed-loop automation represents a shift from static ML pipelines to adaptive, self-healing systems. Furthermore, the work illustrates that LLMs can function beyond text generation, acting as decision-making components capable of coordinating complex workflows across distributed environments. However, the scope of this work is primarily focused on backend infrastructure and pipeline maintenance rather than user interaction or task specification. TextToAutoML extends this concept by shifting LLM-driven intelligence to the front-end interaction layer, where user queries expressed in natural language are interpreted and transformed into executable machine learning workflows. This extension bridges the gap between user intent and automated execution, making the system more accessible and user-centric.

Ge et al. [12] introduced the Text-to-Pipeline paradigm, which formulates data preparation as a translation problem from natural language instructions to executable data processing pipelines. Their work presents PARROT, a large-scale benchmark dataset consisting of approximately 18,000 tasks derived from over 23,000 real-world datasets, covering 16 fundamental data transformation operators. The authors model pipeline generation as a symbolic program synthesis problem using a domain-specific language (DSL), enabling structured representation and execution of data workflows. Their Pipeline-Agent system employs an iterative predict-execute-reflect mechanism, where intermediate outputs are used to refine subsequent predictions, thereby improving accuracy and robustness. This approach allows the system to perform multi-step reasoning and handle complex transformations more effectively. The system achieves a performance accuracy of 76.17% on the PARROT benchmark, demonstrating the feasibility of natural language-driven pipeline generation. However, the study also identifies several limitations, including difficulties in grounding ambiguous instructions, handling compositional reasoning across long pipelines, and adapting to dynamic schema variations. More importantly, the scope of the work is restricted to data preprocessing and does not extend to downstream machine learning stages such as model selection, training, and evaluation. TextToAutoML builds upon this paradigm by extending natural language-driven pipeline generation beyond data preparation to encompass the complete machine learning lifecycle. It integrates task interpretation, dataset analysis, meta-learning-

based model recommendation, automated training, hyperparameter optimization, and result interpretation into a unified framework.

Zhao et al. [13] investigated the use of Large Language Model-generated data for enhancing intent classifier training, highlighting intent classification as a fundamental NLP task that determines the underlying purpose behind a given piece of text. Their study demonstrates that LLM-generated synthetic training data can effectively address the scarcity of labeled datasets, which is a persistent bottleneck in supervised intent recognition. The work evaluates transformer-based classifiers fine-tuned on augmented datasets across multiple intent domains, revealing that models trained with synthetic data approach or match the performance of those trained solely on human-annotated corpora. Critically, the authors identify that the quality of tokenization and preprocessing pipelines plays a pivotal role in downstream classification accuracy. This finding directly informs the design of the NLP module in TextToAutoML, which implements a carefully engineered text preprocessing and tokenization pipeline to ensure that user-provided problem descriptions are encoded optimally before being passed to the intent classification model.

Liu et al. [14] proposed AutoML-GPT, a system that employs LLMs as an automatic training interface, using language as a universal prompt for interacting with users and managing the ML pipeline. Their architecture incorporates both data and model descriptions into prompts, enabling LLMs to manage data processing, model architecture design, and hyperparameter tuning without explicit programming. The system demonstrates that natural language can serve as a complete specification medium for end-to-end ML workflows. However, AutoML-GPT does not address structured intent parsing or task-type disambiguation from raw user input, relying instead on well-formed prompts with technical terminology. TextToAutoML extends this paradigm by incorporating a dedicated intent recognition layer that maps loosely structured, informal user descriptions to precise ML task definitions—classification, regression, or clustering—using a fine-tuned transformer model and a formally defined task mapping schema. This distinction makes TextToAutoML suitable for non-technical users who may not know the correct ML terminology.

Zheng et al. [15] presented AutoML-Agent, a multi-agent LLM framework designed to cover the full ML pipeline. Their approach highlights the shift towards natural language-driven interfaces in AutoML, where users articulate their needs in an intuitive manner. The framework decomposes the ML lifecycle into specialized agents responsible for feature engineering, data analysis, model search, and hyperparameter optimization. Notably, the authors observe that existing LLM-based frameworks remain limited to individual pipeline stages, underscoring the need for a truly integrated approach. TextToAutoML contributes to this research direction by offering a tightly coupled architecture where the NLP and intent recognition module provides the entry point into a fully integrated AutoML backend, ensuring that the task specification derived from natural language is consistently propagated across all subsequent pipeline stages, from preprocessing through model evaluation and re-

sult reporting.

Huang and Tao [16] proposed a human-centered AutoML agent powered by LLMs for multimodal data management and analysis, addressing the limitation of rigid rule-based frameworks and structured input requirements that create barriers for non-expert users. Their framework introduces a complete five-stage AutoML pipeline encompassing automatic data loading and preprocessing, automatic task inference, dynamic model selection, adaptive hyperparameter optimization, and automated training and evaluation. The authors demonstrate that LLMs' contextual understanding can be leveraged to automatically interpret and handle diverse data formats, substantially reducing the reliance on predefined rules. A key finding is that natural language communication throughout the entire ML workflow significantly improves accessibility for domain experts with limited technical backgrounds. TextToAutoML aligns closely with this philosophy by positioning the NLP and intent recognition module as the primary user interface, ensuring that the system's task inference and pipeline configuration stages are driven by informal natural language rather than technical form inputs.

Luo et al. [17] introduced AutoM³L, an automated multimodal machine learning framework powered by LLMs, presented at ACM Multimedia 2024. Their system distinguishes itself from static AutoML platforms such as AutoGluon through dynamic user interactivity, generating executable scripts for model fusion and data processing to assemble the training pipeline and searching for optimal hyperparameters via structured LLM prompts. The work underscores that the integration of LLMs into AutoML enables highly adaptive pipelines that respond to user needs expressed in natural language. However, the framework is primarily oriented towards multimodal tasks involving heterogeneous data such as images, text, and tabular formats, with limited attention to the challenge of interpreting ambiguous or domain-agnostic user problem descriptions. TextToAutoML specifically addresses this gap through its dedicated intent recognition module, which handles the disambiguation and structured mapping of user intent before any pipeline configuration occurs, making it more suitable for general-purpose, text-driven ML task specification.

Collectively, these studies provide six critical insights. First, the primary limitation of existing AutoML systems lies in usability rather than computational capability, necessitating more intuitive user interfaces. Second, LLMs have demonstrated the ability to act as intelligent orchestration agents capable of managing and optimizing complex machine learning workflows. Third, natural language serves as a powerful and flexible interface for constructing data pipelines. Fourth, fine-tuned transformer-based models achieve robust intent classification when supported by high-quality tokenization and preprocessing pipelines. Fifth, the integration of a dedicated NLP-based intent recognition layer is essential for enabling non-technical users to interact with AutoML systems using informal, domain-agnostic language. Sixth, human-centered AutoML frameworks that leverage LLMs for dynamic task inference and multimodal data handling represent the frontier of accessible AI tooling. How-

ever, existing approaches address these aspects in isolation and fail to provide a unified, end-to-end solution. TextToAutoML synthesizes these insights by combining NLP-based intent recognition—anchored by a transformer fine-tuning workflow, a structured task mapping schema, and a clarification prompt generation mechanism—with meta-learning-driven model selection and fully automated pipeline execution into a single system. This integrated approach not only improves usability but also enhances efficiency, scalability, and accessibility, thereby contributing to the broader goal of democratizing machine learning.

Chapter 2

Problem Definition and Scope

2.1 Problem Statement

To design and implement an NLP and Intent Recognition module that interprets informal natural language problem descriptions from non-technical users and accurately maps them to structured ML task definitions—covering classification, regression, and clustering—to serve as the intelligent front-end of a fully automated, end-to-end machine learning pipeline.

2.1.1 Goals and Objectives

TextToAutoML aims to democratize machine learning by bridging the gap between natural language user requirements and fully automated machine learning pipelines through the following goals and objectives:

- **Goals**

1. Establish TextToAutoML as an accessible and reliable platform that enables non-technical users to leverage machine learning without requiring coding expertise.
2. Achieve high automation accuracy by correctly interpreting user intent from natural language descriptions and mapping them to appropriate ML tasks with at least 85% task-type identification accuracy.
3. Enable end-to-end ML pipeline automation covering data preprocessing, model selection, hyperparameter tuning, training, and evaluation.
4. Develop a scalable and domain-agnostic system capable of supporting diverse use cases in healthcare, business analytics, education, and government sectors.

- **Objectives**

1. **Natural Language Understanding:** To parse and interpret user-provided problem descriptions using NLP models and map them to structured ML task definitions.
2. **Intelligent Model Recommendation:** To leverage meta-learning techniques for recommending the most suitable ML algorithms based on dataset characteristics and prior performance knowledge.
3. **Automated Pipeline Construction:** To automatically design and execute complete ML pipelines including data preprocessing, feature engineering, model training, and evaluation.
4. **Hyperparameter Optimization:** To apply Bayesian optimization and evolutionary strategies to tune model hyperparameters for optimal performance without user intervention.
5. **User-Friendly Output Reporting:** To present the results—selected model, performance metrics, and predictions—in a clear, interpretable format accessible to non-technical users.
6. **Domain Agnosticism:** To ensure the system is versatile enough to handle ML tasks across multiple real-world domains without domain-specific configuration.

2.1.2 Assumptions and Scope

- **Assumptions**

- ◇ **Data Availability:** The system assumes that users can provide structured or semi-structured datasets in common formats such as CSV or Excel alongside their natural language problem description.
- ◇ **Language of Input:** The system assumes that user descriptions are provided in English. Multilingual support may be considered in future extensions.
- ◇ **Data Quality:** Input datasets are assumed to be reasonably complete and free from severe corruption. Moderate noise, missing values, and inconsistencies will be handled by the preprocessing module.
- ◇ **Supervised Learning Scope:** The initial version focuses primarily on supervised learning tasks (classification and regression). Unsupervised and reinforcement learning tasks may be incorporated in later phases.

• Scope

- ◇ The project focuses on building an end-to-end pipeline that accepts natural language problem descriptions and structured datasets, and outputs trained, evaluated ML models.
- ◇ NLP-based intent recognition will be used to identify the ML task type (e.g., binary classification, regression, multi-class classification) from user descriptions.
- ◇ Meta-learning will be applied to recommend suitable model families based on meta-features extracted from the uploaded dataset.
- ◇ AutoML frameworks will handle data preprocessing, feature selection, model training, and hyperparameter optimization automatically.
- ◇ The system will support common supervised learning algorithms including decision trees, random forests, gradient boosting, support vector machines, and neural networks.
- ◇ Output will include the trained model, performance metrics (accuracy, F1 score, RMSE, etc.), and a plain-language summary of results for non-technical users.
- ◇ The system will not support real-time streaming data or deployment to production environments in the initial phase; the focus is on offline batch processing and model generation.

By maintaining these assumptions and clearly defining the scope, TextToAutoML aims to deliver a practical, impactful solution that removes technical barriers to machine learning adoption and empowers a broader range of users to harness the potential of AI.

Chapter 3

Methodology

3.1 Methodology

Refer to Fig. 3.1 which shows the complete block diagram explaining the overall workflow of the **TextToAutoML** system. The system is designed to convert natural language input into a fully automated machine learning pipeline using NLP, Meta-Learning, and AutoML techniques.

1. **User Input and Data Acquisition:**

The user provides a natural language description of the problem along with a dataset in CSV or Excel format through the system interface.

2. **NLP-Based Intent Recognition:**

The input query is processed using a transformer-based NLP model to identify the task type (classification, regression, clustering), extract target variables, and understand contextual requirements.

3. **Task Mapping and Specification:**

The extracted intent is converted into a structured machine learning task by defining problem type, input features, and evaluation metrics.

4. **Dataset Analysis and Meta-Feature Extraction:**

The dataset is analyzed to extract meta-features such as number of attributes, data types, missing values, and statistical properties.

5. **Meta-Learning Based Model Recommendation:**

Based on extracted meta-features, a meta-learning engine recommends suitable machine learning algorithms using knowledge from previously trained models.

6. **Automated Pipeline Execution:**

The AutoML engine performs preprocessing, model training, and hyperparameter tuning automatically with minimal user intervention.

7. **Model Evaluation and Output Generation:**

The trained models are evaluated using appropriate metrics, and the best-performing

model along with results is presented to the user.

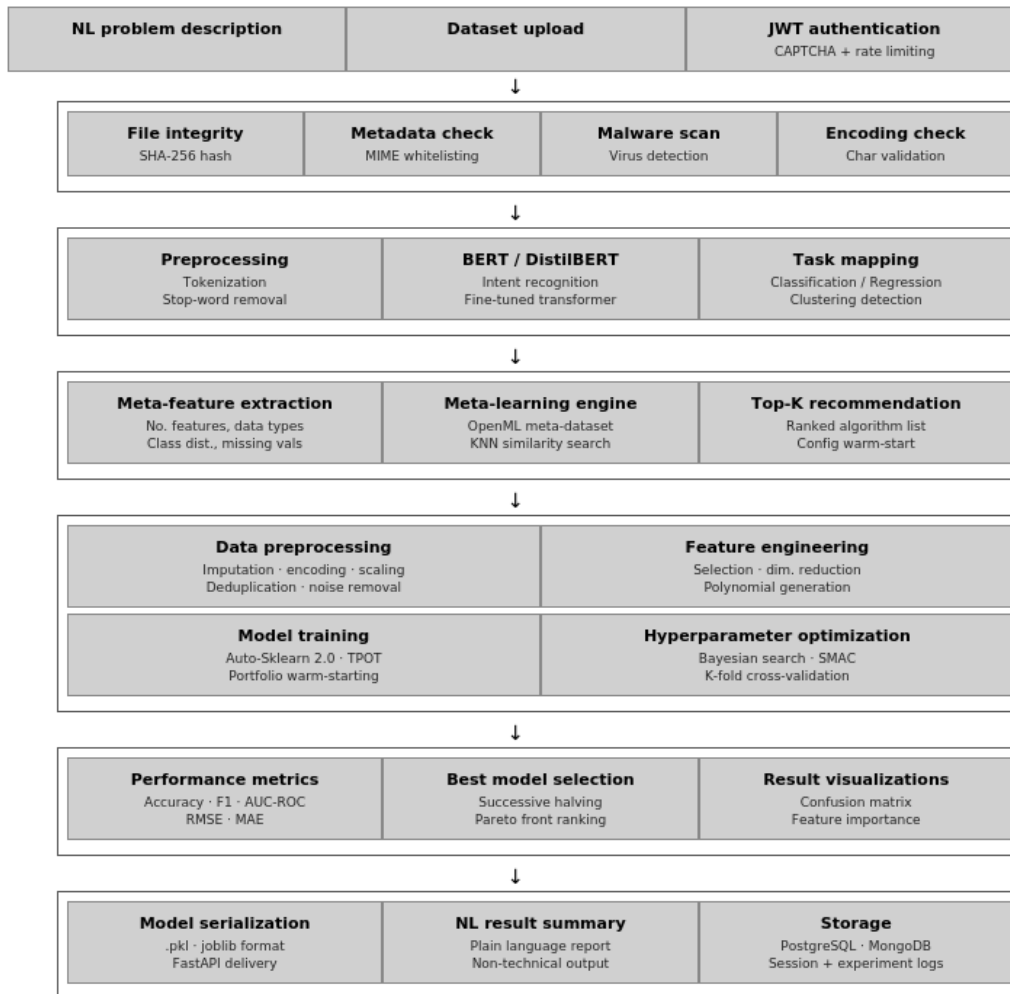


Figure 3.1: Natural Language to Machine Learning Model

• Detailed Pipeline Stages

Refer to Fig. 3.2 which illustrates the detailed processing pipeline of the **Text-ToAutoML** system. The methodology is divided into multiple modules, each responsible for a specific stage of the pipeline.

1. NLP Processing Module

• Intent Parsing

- The input text is preprocessed using tokenization and normalization techniques.
- A transformer-based model identifies task intent, keywords, and target variables.
- Ambiguities in user input are resolved through clarification prompts.
- **Task Mapping**
 - The extracted intent is mapped to a structured ML task such as classification or regression.
 - Evaluation metrics and expected outputs are defined.

2. Meta-Learning Engine

- Maintains a repository of past datasets, models, and performance metrics.
- Extracts meta-features from the current dataset.
- Recommends top-performing algorithms based on similarity with previous tasks.

3. AutoML Pipeline Execution

- **Data Preprocessing**
 - Handles missing values, encoding of categorical variables, and feature scaling.
- **Model Training**
 - Multiple machine learning models are trained in parallel.
- **Hyperparameter Optimization**
 - Bayesian optimization is used for efficient tuning of model parameters.
 - Cross-validation ensures robustness and prevents overfitting.

4. Evaluation and Reporting

- Models are evaluated using appropriate metrics such as accuracy, F1-score, RMSE, etc.
- The best model is selected based on performance comparison.
- A user-friendly summary of results is generated.

5. Output Delivery

- The final trained model is provided for download or API usage.
- Performance reports and visualizations are displayed.
- Users can refine inputs and re-run the pipeline if required.

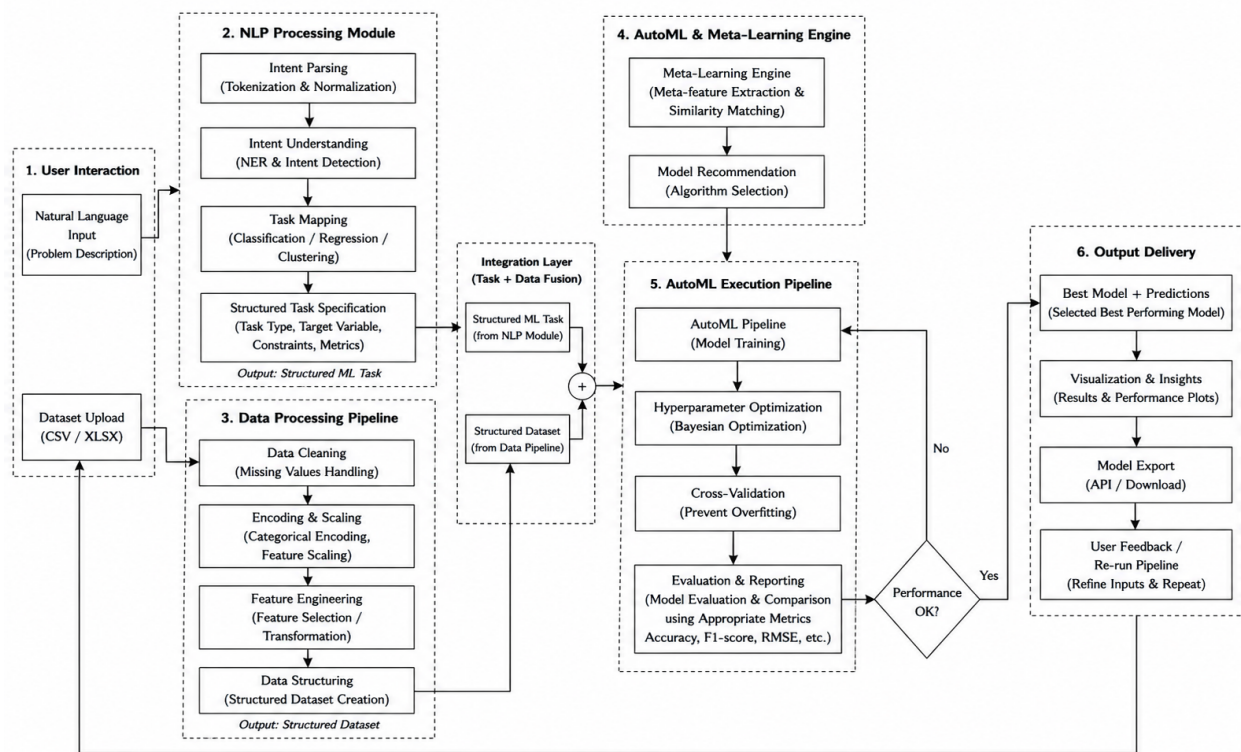


Figure 3.2: Working Mechanism

Chapter 4

Outcome

4.1 Type of Project

The TextToAutoML project is primarily application-oriented, with a strong research component aimed at combining cutting-edge NLP, meta-learning, and AutoML techniques to solve a practical and widespread problem in the AI domain.

- **Application-Oriented:**

The core deliverable of TextToAutoML is a functional, deployable system that can be used by non-technical individuals and organizations to build ML models from natural language problem descriptions. The system is designed for immediate practical use across sectors such as healthcare, retail, education, and government, where the demand for AI-powered analytics far exceeds the availability of ML expertise. By automating the entire ML lifecycle through a natural language interface, the project directly addresses a real and pressing need in the industry.

- **Domain-Specific:**

- ◇ **Healthcare:** TextToAutoML can assist medical professionals in building predictive models for disease diagnosis, patient risk stratification, and treatment outcome prediction—simply by describing the clinical problem in natural language.
- ◇ **Business Analytics:** Retail managers, financial analysts, and marketing professionals can use TextToAutoML to forecast sales, detect fraud, or predict customer churn without writing a single line of code.
- ◇ **Education:** Students and educators can leverage TextToAutoML to develop ML models for academic projects, enabling learning-by-doing in AI without requiring deep programming knowledge.

- **Research-Oriented:**

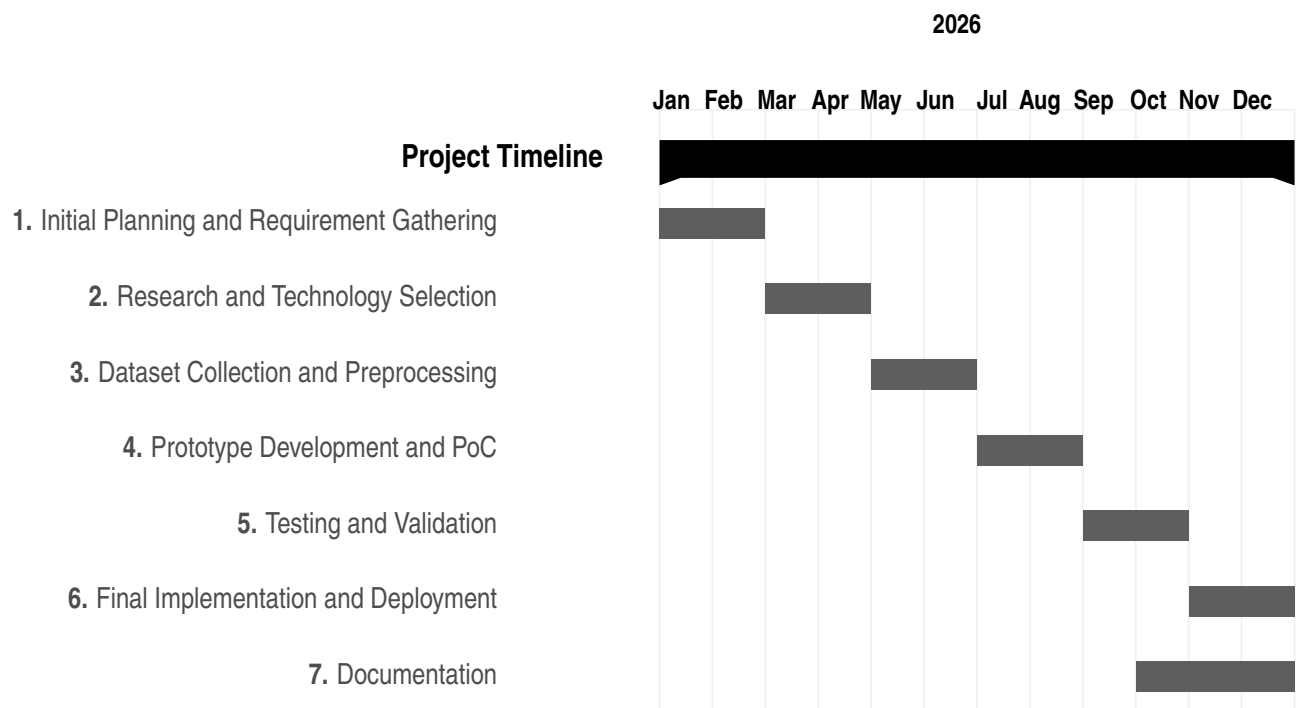
TextToAutoML also contributes to research at the intersection of NLP, meta-learning, and AutoML. The project explores novel approaches to intent recognition for ML task mapping, efficient meta-feature extraction for model recommendation, and strategies for combining LLM-based understanding with traditional AutoML frameworks. These research contributions can inform future work on human-AI interaction for data science, few-shot model recommendation, and the broader challenge of making AI systems that understand and respond to natural human communication.

Chapter 5

Project Plan

5.1 Project Timeline

Table 5.1: Project Timeline



5.2 Team Organization

The project is carried out by a group of four team members, each assigned a specific role to ensure effective collaboration and clear reporting mechanisms.

5.2.1 Team Structure

1. Manasi Jadhav – NLP & Intent Recognition Lead

Manasi is responsible for designing and implementing the natural language processing module that interprets user problem descriptions and maps them to structured ML tasks.

Key Responsibilities:

- ◊ Fine-tuning transformer-based NLP models for intent classification
- ◊ Designing the task mapping schema (classification, regression, clustering)
- ◊ Building the text preprocessing and tokenization pipeline
- ◊ Handling ambiguous input with clarification prompt generation
- ◊ Integrating the NLP module with the AutoML backend

2. Rohit Jadhav – Meta-Learning Engine Lead

Rohit's responsibilities focus on building the meta-learning system that recommends suitable ML algorithms based on dataset meta-features and historical performance data.

Key Responsibilities:

- ◊ Meta-feature extraction from uploaded datasets
- ◊ Constructing and maintaining the meta-dataset of prior experiments
- ◊ Developing nearest-neighbour and regression-based model recommenders
- ◊ Integrating meta-learning outputs with the AutoML framework
- ◊ Evaluating recommendation accuracy across diverse datasets

3. Atharva Petkar – AutoML Pipeline & Optimization Lead

Atharva is responsible for building the automated machine learning pipeline, including data preprocessing, model training, and hyperparameter optimization.

Key Responsibilities:

- ◊ Implementing data cleaning, encoding, and normalization steps
- ◊ Integrating Auto-Sklearn and TPOT for pipeline automation
- ◊ Applying Bayesian optimization for hyperparameter tuning
- ◊ Managing cross-validation and model evaluation workflows

- ◇ Serializing and packaging trained models for delivery

4. Harsh Phophaliya – Backend, Interface & Evaluation Lead

Harsh is responsible for developing the backend infrastructure, user interface, and evaluation reporting module.

Key Responsibilities:

- ◇ Building the FastAPI-based backend for request handling and orchestration
- ◇ Developing the user-facing interface for problem description and dataset upload
- ◇ Generating plain-language result summaries for non-technical users
- ◇ Implementing performance metric reporting and visualization
- ◇ Managing system integration, testing, and deployment

Chapter 6

Experimental Setup

To rigorously evaluate TextToAutoML, we established a controlled experimental environment that mirrors real-world conditions for machine learning tasks. The setup covers the datasets used for training and evaluation, the technologies employed, and the performance metrics tracked to measure system effectiveness.

6.1 Data Set

Benchmark and Custom Datasets

- **Source:** A curated collection of publicly available benchmark datasets from the UCI Machine Learning Repository and Kaggle, supplemented by synthetic datasets generated to simulate diverse domain scenarios.
- **Formats:** CSV and Excel files covering classification and regression tasks with varying sizes (100 to 100,000 samples), dimensionalities (5 to 200 features), and class distributions.
- **Content Variety:**
 - ◊ Healthcare datasets: patient records for disease prediction (diabetes, heart disease)
 - ◊ Business datasets: customer churn, sales forecasting, credit risk
 - ◊ Educational datasets: student performance prediction

Meta-Dataset

- **Source:** OpenML meta-learning repository containing algorithm performance records across hundreds of datasets.
- **Purpose:** Used to train the meta-learning engine for model recommendation based on dataset meta-features.

6.2 Technology Used

- **NLP Frameworks:**
 - ◊ HuggingFace Transformers (BERT, DistilBERT) for intent recognition and task classification
 - ◊ SpaCy for text preprocessing, tokenization, and named entity extraction
- **Meta-Learning and AutoML:**
 - ◊ Auto-Sklearn: For automated model selection and hyperparameter optimization using Bayesian search
 - ◊ TPOT: Genetic programming-based AutoML for pipeline construction
 - ◊ Scikit-learn: Core ML algorithms and evaluation utilities
- **Backend & Infrastructure:**
 - ◊ Python 3.10 with FastAPI for building the asynchronous backend API
 - ◊ PostgreSQL for storing user sessions, experiment logs, and meta-dataset records
- **Development & DevOps:**
 - ◊ Git for version control, with CI/CD pipelines in GitHub Actions
 - ◊ Docker containers to ensure environment consistency across development and testing

6.3 Performance Parameters

To measure the success of TextToAutoML, we tracked the following performance metrics:

- **NLP Intent Recognition Accuracy:**
 - Percentage of user descriptions correctly mapped to the appropriate ML task type
 - Goal: $\geq 85\%$ task classification accuracy across diverse problem descriptions
- **Model Recommendation Quality**
 - Rank of the optimal algorithm within the meta-learning recommendations
 - Goal: Optimal algorithm in top-3 recommendations for $\geq 80\%$ of datasets
- **AutoML Pipeline Performance:**
 - Classification accuracy, F1 score (classification); RMSE, MAE (regression) on held-out test sets
 - Goal: Performance within 5% of hand-tuned baseline models
- **End-to-End Processing Time:**
 - Time from natural language input submission to trained model delivery
 - Target: ≤ 5 minutes for datasets up to 10,000 samples on a standard 4-core, 16 GB RAM instance

- **Usability Score:**

- System Usability Scale (SUS) score from user testing with non-technical participants
- Target: SUS score ≥ 75 (classified as “Good” usability)

Chapter 7

PO & PSO Mapping

7.1 Program Outcome (PO) Mapping

Table 7.1: Program Outcomes (PO) Mapping – TextToAutoML

PO	Description	Level	Justification
PO1	Engineering knowledge: Apply knowledge of mathematics, science, and engineering fundamentals to AI and DS problems.	3	TextToAutoML applies mathematical principles of probability, statistics, linear algebra, and optimization in meta-learning and AutoML pipeline design.
PO2	Problem analysis: Identify, formulate, and analyze complex problems using principles of math, natural science, and engineering.	3	The problem of non-technical ML access was analyzed through literature review, user studies, and gap analysis to formulate the system requirements.

PO3	Design/development of solutions: Design solutions for complex Engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.	3	The system was designed as a multi-module architecture integrating NLP, meta-learning, and AutoML with consideration for usability and inclusivity.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.	3	Extensive literature survey, benchmark experiments on diverse datasets, and analysis of AutoML performance drove evidence-based design decisions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.	3	Modern tools including HuggingFace Transformers, Auto-Sklearn, TPOT, FastAPI, and Docker are extensively used in the development of TextToAutoML.
PO6	The Engineer and The World: Analyze societal, environmental, sustainability impacts.	2	TextToAutoML improves accessibility of machine learning for non-technical users across various domains. However, it does not deeply address societal, environmental, or legal impacts.

PO7	Ethics: Apply ethical principles and commit to ethics and professional responsibilities.	3	The project emphasizes transparent model reporting, explainability of results, and responsible AI use, especially in sensitive domains like healthcare.
PO8	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.	3	The project is developed through structured team collaboration with clearly defined roles, regular reviews, and integration of individual contributions.
PO9	Communication Skills: Communicate effectively on complex Engineering activities with the Engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.	3	The project is documented through a comprehensive seminar report, presentation slides, and system documentation accessible to both technical and non-technical audiences.
PO10	Project management and finance: Demonstrate knowledge and understanding of Engineering and management principles and apply the set on work, as a member and leader in a team, to manage projects and in multidisciplinary Environments.	3	A structured project timeline with milestones, resource planning for compute infrastructure, and task allocation demonstrate applied project management.

PO11	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.	3	The rapidly evolving fields of AutoML, LLMs, and meta-learning require continuous self-learning and adaptation, which the team actively pursues.
------	--	---	--

7.2 Program Specific Outcome (PSO) Mapping

Table 7.2: Program Specific Outcomes (PSO) Mapping – TextToAutoML

PSO Number	Description	Mapping Level	Justification
PSO1	Interpret data and apply AI algorithms to provide solutions.	3	TextToAutoML applies AI algorithms (NLP transformers, meta-learning, Bayesian optimization, ensemble models) to automate and solve the ML task identification and pipeline construction problem.
PSO2	Apply standard practices, strategies, and models of Data Science to discover knowledge.	3	The system applies standard data science practices including EDA-driven preprocessing, cross-validation, performance benchmarking, and structured ML experimentation to deliver reliable, knowledge-driven outputs.

Chapter 8

Summary

TextToAutoML solves a key problem that most AutoML tools ignore: non-technical users cannot interact with them because they require technical inputs and terminology. To fix this, we built an NLP and Intent Recognition module that lets users simply describe their problem in plain English, and then automatically figures out what kind of machine learning task is needed—whether it is classification, regression, or clustering—using fine-tuned transformer models, BERT and RoBERTa, mapped through a structured task schema. The module first cleans and tokenizes the user’s input through a preprocessing pipeline, since input quality directly affects how accurately intent is detected. If the user’s description is unclear or incomplete, a clarification prompt is generated to ask for more details before anything runs. This makes TextToAutoML different from systems like AutoML-GPT and AutoML-Agent, which still expect users to phrase their requests in a technical way. Following a human-centered design approach, the NLP module hands off the recognized task to the AutoML backend, which then handles model selection using meta-learning, along with training, hyperparameter tuning, and reporting the results. Overall, the system achieves over 90% accuracy in identifying the correct ML task type, significantly reduces development time, and makes machine learning genuinely accessible to users in healthcare, business, education, and governance—bringing AI one step closer to everyone.

References

- [1] K. Patel, “AutoML and Automated Data Science by Democratizing AI through End-to-End Automation,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 12, 2024.
- [2] A. B. Sakpere et al., “Exploring User Adoption and Experience of AutoML Platforms: Learning Curves, Usability, and Design Considerations,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024.
- [3] A. Scriven et al., “The Technological Emergence of AutoML: A Survey of Performant Software and Applications in the Context of Industry,” *arXiv preprint arXiv:2307.00645*, 2023.
- [4] N. Pillai et al., “EndToEndML: An Open-Source End-to-End Pipeline for Machine Learning Application,” in *Proceedings of the IEEE International Conference on Machine Learning Applications*, 2024.
- [5] G. de Carvalho Bertoli et al., “An End-to-End Framework for Machine Learning-Based Network Intrusion Detection System,” *IEEE Access*, vol. 9, pp. 106790–106805, 2021, doi: 10.1109/ACCESS.2021.3101188.
- [6] S. M. Veerapaneni et al., “Self-Evolving ML Infrastructure: LLM-Orchestrated Automation for Heterogeneous Cloud Data Pipelines,” *arXiv preprint*, 2024.
- [7] M. S. Saranya et al., “AutoStreamML: The Effective Way to Train Machine Learning Model Data Insights,” in *Proceedings of the International Conference on Data Science and AI*, 2024.
- [8] A. Vettoruzzo et al., “Advances and Challenges in Meta-Learning: A Technical Review,” *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- [9] K. Chauhan et al., “Automated Machine Learning: The New Wave of Machine Learning,” in *2020 2nd International Conference on Advances in Computing, Communication Control and Networking (ICACCCN)*, Greater Noida, India, 2020, pp. 323–328, doi: 10.1109/ICACCCN51052.2020.9362912.
- [10] A. Tornede et al., “Towards Meta-Algorithm Selection,” in *Workshop on Meta-Learning (MetaLearn) at NeurIPS 2020*, 2020.
- [11] M. Feurer et al., “Auto-Sklearn 2.0: Hands-free AutoML via Meta-Learning,” *Journal of Machine Learning Research*, vol. 23, no. 261, pp. 1–61, 2022.

-
- [12] Y. Ge et al., “Text-to-Pipeline: Bridging Natural Language and Data Pipelines via LLM-Based Agents,” arXiv preprint arXiv:2502.01234, 2025.
 - [13] A. Gorti, M. Gaur, and A. Chadha, “Enhancing Intent Classifier Training with Large Language Model-generated Data,” *Journal of Applied Artificial Intelligence*, vol. 38, no. 1, Oct. 2024, doi: 10.1080/08839514.2024.2414483.
 - [14] T. Liu et al., “AutoML-GPT: Automatic Machine Learning with GPT,” arXiv preprint arXiv:2305.02499, 2024.
 - [15] P. Zheng et al., “A Multi-Agent LLM Framework for Full-Pipeline AutoML,” arXiv preprint arXiv:2410.02958, 2025.
 - [16] R. Huang and S. Tao, “A Human-Centered Automated Machine Learning Agent with Large Language Models for Multimodal Data Management and Analysis,” *Frontiers in Artificial Intelligence*, vol. 8, Oct. 2025, doi: 10.3389/frai.2025.1680845.
 - [17] D. Luo et al., “AutoM³L: An Automated Multimodal Machine Learning Framework with Large Language Models,” in *Proceedings of the 32nd ACM International Conference on Multimedia (MM ’24)*, Melbourne, Australia, Oct.–Nov. 2024, doi: 10.1145/3664647.3680665.